

Smart AI Attendance

Developer Guide

A-Z file reference with architecture, logic, and changes

Version 1.1 | 2026

1. Architecture Overview

The system has four layers: Edge Node (camera AI), Backend API (FastAPI), Frontend Dashboard (Next.js 15), and Infrastructure (Postgres, Redis, Nginx, Celery).

Data flow

```
Camera (RTSP/IP)
-> Edge Node: YOLO detect -> ByteTrack -> face crop -> anti-spoof -> ArcFace embed -> FAISS match
-> POST /api/v1/attendance/event [EDGE_TOKEN auth]
-> Backend: confidence gate -> cooldown check (Redis) -> shift calc -> Postgres write
    -> fire alerts if needed -> Redis pub/sub publish
-> WebSocket server -> Next.js client (real-time feed)
```

Technology stack

Layer	Technology	Why
Edge AI	YOLOv11 + ByteTrack	Fast multi-object detection + persistent track IDs
Face AI	InsightFace ArcFace R100	512-d embeddings, best accuracy/speed tradeoff
Vector search	FAISS (edge) + pgvector (DB)	Sub-ms local search, persistent storage in Postgres
Backend	FastAPI + asyncpg	Async I/O for Redis, DB, and Slack without blocking
DB	PostgreSQL 16 + pgvector	ACID + native vector extension
Cache/broker	Redis 7	Cooldown TTLs, pub/sub, Celery broker
Frontend	Next.js 15 App Router	SSR/SSG, TypeScript, Tailwind dark mode
Auth	JWT HS256 + bcrypt	Stateless, no DB lookup on each request
GPU	ORT auto-detect	NVIDIA/AMD/Intel/CPU selected at runtime

Multi-tenancy

Every DB table has a tenant_id FK. The JWT payload carries tenant_id. Every query filters by user.tenant_id from the JWT payload -- no cross-tenant data leakage is possible without a compromised JWT secret.

2. Backend Files

backend/app/config.py

Application settings loaded from .env via pydantic-settings

Uses pydantic-settings BaseSettings. All settings are read from environment variables (or .env file). The validate_production_secrets() method is called at startup and raises RuntimeError if SECRET_KEY, FACE_ENCRYPTION_KEY, or EDGE_TOKEN are still set to their insecure defaults when ENVIRONMENT=production.

SECURITY Added EDGE_TOKEN field + validate_production_secrets() startup guard

- SECRET_KEY: signs JWT tokens (HS256). Must be 64+ random hex chars in production.
- FACE_ENCRYPTION_KEY: Fernet key for face embeddings at rest.
- EDGE_TOKEN: shared secret authenticating edge-node event POSTs.
- CONFIDENCE_THRESHOLD: minimum face match cosine similarity (0.75 dev, 0.82 prod).

backend/app/main.py

FastAPI application factory: middleware, lifespan, route registration

Creates the FastAPI app, registers all middleware, mounts all routers, and manages the Redis connection lifecycle via @asynccontextmanager lifespan.

SECURITY Swagger/Redoc disabled in ENVIRONMENT=production

SECURITY Rate limit lowered from 1000 to 200 rpm; login/refresh capped at 30 rpm

NEW Added shifts.router and admin.router

- TenantMiddleware: reads X-Tenant-ID header into request.state.tenant_id.
- RateLimitMiddleware: per-IP sliding window, separate tighter limit for auth paths.
- CORSMiddleware: origins from settings.ALLOWED_ORIGINS.

backend/app/dependencies.py

FastAPI dependency injectors used across all routes

Contains: get_db (yields AsyncSession), get_redis (lazy reconnect), get_current_user (validates JWT), role_required (RBAC gate), verify_edge_token (HMAC edge auth).

BUG FIX Removed duplicate get_current_user + role_required definitions

SECURITY Added verify_edge_token using secrets.compare_digest (timing-safe)

```
async def verify_edge_token(authorization: Header) -> None:
    if not settings.EDGE_TOKEN: return # dev: skip check
    if not secrets.compare_digest(token_str, settings.EDGE_TOKEN):
        raise HTTP 401
```

backend/app/core/security.py

JWT creation/decoding, password hashing, role hierarchy

ROLE_HIERARCHY maps role names to integers (super_admin=6 .. viewer=1). require_role(user_role, minimum) returns True if user_role level >= minimum level. create_access_token embeds sub, tenant_id, role, exp, iat into the JWT payload.

- hash_password / verify_password: bcrypt via passlib CryptContext.
- decode_token: jose jwt.decode with HS256 -- validates signature and expiry without DB call.

backend/app/core/middleware.py*Tenant context injection and rate limiting***CHANGE** Auth paths capped at 30 rpm (was 10, which was too low for normal use)**BUG FIX** _cleanup() added to purge stale IP buckets every 5 minutes (memory leak fix)

RateLimitMiddleware stores timestamps per IP in a dict. The window is 60 seconds (sliding). The cleanup method removes IP entries idle > 60 s every 5 minutes.

Models (backend/app/models/)

SQLAlchemy 2.0 ORM with Mapped[] type annotations. Every model inherits UUIDMixin (auto UUID PK) and TimestampMixin (created_at).

backend/app/models/shift.py*Shift and EmployeeShift -- core of late/early/OT logic*

Shift stores: name, start_time, end_time, grace_in_min (late threshold), early_out_min (early leave threshold), work_days (int array). EmployeeShift is a composite-PK join table with effective_from/to dates.

NEW New model (shifts feature)**backend/app/models/alert_config.py***Per-tenant configurable thresholds***BUG FIX** confidence_threshold and liveness_threshold changed from Mapped[float] to Mapped[int] -- stored as 0-100 integer, divided**backend/app/models/user.py***User accounts and audit logs*

User has: tenant_id, email (unique), hashed_password, full_name, role, is_active, last_login_at. AuditLog records every action with ip_address (INET type), user_agent, old_values, new_values (JSON).

Services (backend/app/services/)**backend/app/services/attendance_service.py***Main business logic for event processing*

process_recognition_event() is the core function. It runs in this order:

1. Liveness gate: spoof detected? Fire alert, skip.
2. Restricted camera check: fire alert if is_restricted=True.
3. Confidence gate: below threshold? Handle as unknown person.
4. Employee lookup + active check.
5. Cooldown check (Redis key: cooldown:{emp_id}:{cam_id}, TTL=COOLDOWN_MINUTES).
6. Blacklist + VIP alert firing.
7. Shift lookup for the employee today.
8. After-hours check (compares now_local vs shift window).
9. Loitering check (Redis dwell timer).

10. Get or create AttendanceLog for today.
11. Determine action: check_in, check_out (exit camera), or skip_duplicate.
12. Calculate late_by_min (check-in after start+grace) or early_by_min (check-out before end-buffer).
13. Commit to DB, set cooldown key, publish to Redis pub/sub.

BUG FIX	Removed dead import pytz as _tz
BUG FIX	clear_loitering made async (was sync calling asyncio.create_task)
BUG FIX	UUID parsing in _handle_unknown wrapped in try/except

backend/app/services/alert_service.py

Alert firing and loitering dwell tracking

fire() creates an Alert DB record, publishes to Redis, and sends Slack for VIP/blacklist. check_loitering() stores a Redis key with first-seen timestamp; fires when elapsed $\geq$ LOITERING_THRESHOLD_MIN. clear_loitering() is called on check-out to reset the dwell timer.

API Routes (backend/app/api/v1/)

backend/app/api/v1/auth.py

Login, token refresh, password change

NEW New: POST /change-password -- verifies current password before accepting new

change_password() loads the User by JWT sub, verifies current_password with bcrypt, hashes new_password, saves.

backend/app/api/v1/enrollment.py

Face embedding enrollment and export

BUG FIX CRITICAL BUG FIX: /match and /export moved before /{employee_id} -- FastAPI was matching 'match' as an employee UUID

backend/app/api/v1/shifts.py

Shift CRUD + employee assignment -- NEW file

NEW New file -- all shift management endpoints

GET /shifts, POST /shifts, PATCH /shifts/{id}, DELETE /shifts/{id}. GET /assignments, POST /assignments (closes previous open assignment first), DELETE /assignments/{emp_id}.

backend/app/api/v1/admin.py

User management -- NEW file

NEW New file -- user CRUD for admin role

GET/POST/PATCH/DELETE /admin/users. Cannot change own role. Checks tenant_id on every user lookup.

backend/app/api/v1/alerts.py

Alert listing, acknowledgement, config

BUG FIX	Missing Unauthorized import added
SECURITY	GET /config changed to require admin role (was accessible to all)
SECURITY	POST /config/password: moved password from query param to request body; requires old password; requires admin role

BUG FIX ChangePasswordIn renamed to AdminConfigPasswordIn to avoid collision with auth.py

backend/app/api/v1/attendance.py

Event ingestion and log management

SECURITY POST /event now requires EDGE_TOKEN (verify_edge_token dependency)

SECURITY PATCH/DELETE /logs/{id} changed from get_current_user to role_required('hr')

SECURITY DELETE /logs (bulk reset) changed to role_required('admin')

backend/app/api/v1/analytics.py

Occupancy, hourly, shift compliance, visitors

BUG FIX Removed dead imports RecognitionEvent, Camera

backend/app/api/v1/rbac.py

Audit log endpoints

SECURITY All audit endpoints changed from get_current_user to role_required('admin') -- audit logs contain IP addresses and action hi

SECURITY GET /audit/actions endpoint now requires admin auth (was unauthenticated)

Schemas (backend/app/schemas/)

backend/app/schemas/attendance.py

Pydantic request/response models for attendance

BUG FIX Added early_by_min: int = 0 to AttendanceLogOut -- model had field, schema didn't

backend/app/schemas/alert_config.py

Alert configuration schemas

BUG FIX Renamed ChangePasswordIn to AdminConfigPasswordIn to avoid collision with auth.py ChangePasswordIn

backend/app/schemas/auth.py

Auth request/response schemas

NEW Added ChangePasswordIn (current_password, new_password) for the change-password endpoint

3. Frontend Files

frontend/src/lib/rbac.ts

Role hierarchy + React hook for client-side RBAC -- NEW file

NEW New file

LEVELS dict mirrors backend ROLE_HIERARCHY exactly. hasRole(minimum) is a pure function -- reads localStorage synchronously. useRole() is a React hook that initialises to 'viewer' (safe for SSR) then sets the real role in useEffect after mount. This fixes the hydration bug where hasRole() called during SSR always returned false (window undefined), causing role-gated buttons to never render.

```
export function useRole() {
  const [role, setRole] = useState('viewer');
  useEffect(() => { setRole(getRole()); }, []);
  const can = (min) => (LEVELS[role] ?? 0) >= (LEVELS[min] ?? 99);
  return { role, can };
}
```

frontend/src/lib/api.ts

All API calls with auto-refresh on 401

request() fetches the API, intercepts 401, tries tryRefresh() once, redirects to login if refresh fails. requestBlob() is the same for file downloads. All API methods are typed with TypeScript generics.

NEW Added changePassword, shift management methods, adminUsers CRUD

frontend/src/types/index.ts

TypeScript interfaces matching backend response shapes

BUG FIX Removed start_time and end_time from Camera interface -- backend never returns these fields

frontend/src/components/ui/DashboardShell.tsx

Sidebar layout shared by all dashboard pages

NEW Added useRole() hook -- role badge, Admin nav link (visible to admin+ after hydration)

NEW Added ChangePasswordModal with current/new/confirm fields, show/hide toggle, bcrypt verified server-side

CHANGE ThemeToggle moved into sidebar footer

NEW Active route highlight using usePathname()

BUG FIX Full dark mode classes throughout

frontend/src/components/live/CameraFeed.tsx

Shared MJPEG camera stream component -- NEW file

NEW New file -- extracted from cameras/page.tsx for reuse on overview dashboard

Shows MJPEG stream with retry-on-failure (5 s). streamOnly prop renders just the video area without card wrapper, used inside cameras/page.tsx cards.

frontend/tailwind.config.js

Tailwind configuration

BUG FIX CRITICAL: Added darkMode: 'class' -- without this, NO dark: variants were generated at all

frontend/src/app/layout.tsx

Root Next.js layout

BUG FIX Removed className='light' from <body> -- was overriding ThemeContext's dark class on <html>

frontend/src/app/globals.css

Global CSS

BUG FIX Added .dark body { background: #111827; color: #f9fafb; } for dark mode body

frontend/src/app/dashboard/page.tsx

Overview / home page

NEW Added Live Cameras section -- loads api.cameras() and shows CameraFeed grid

BUG FIX Replaced hasRole() with useRole() hook to fix SSR hydration issue

frontend/src/app/dashboard/shifts/page.tsx

Shift management page -- NEW file

NEW New file -- create/edit shifts with live threshold preview; assign employees

The modal shows a live summary: 'Late after HH:MM' and 'Early leave before HH:MM' calculated from form values in real time.

frontend/src/app/dashboard/admin/page.tsx

User management page -- NEW file

NEW New file -- user list with role badges, edit-role modal, add-user modal

frontend/src/app/dashboard/employees/page.tsx

Employee management

NEW Bangladesh-friendly autocomplete via <datalist> for Department (17 options) and Designation (50 options)

CHANGE Phone placeholder changed to +880 1XXX-XXXXXX format

SECURITY Role guards: Add/Edit/Enroll/Deactivate require hr+; Blacklist/VIP require admin+

frontend/src/app/dashboard/cameras/page.tsx

Camera management

BUG FIX Removed start_time/end_time fields from form -- backend Camera model never had these columns

SECURITY Role guards: Add/Edit/Delete require admin+

NEW Empty state when no cameras registered, showing contextual message per role

frontend/src/app/login/page.tsx

Login page

SECURITY Removed hardcoded default credentials (admin@demo.com / admin123)

BUG FIX Error message now distinguishes 429 rate-limit from 401 wrong credentials from network error

frontend/src/components/EmployeeEnrollment.tsx

DELETED -- was never imported anywhere

BUG FIX Deleted -- dead code, completely unused component

4. Edge Node Files

edge/src/utils/gpu.py

Runtime GPU auto-detection -- NEW file

NEW

New file -- single source of truth for ORT providers and PyTorch device

detect() is called once at startup and cached. It queries onnxruntime.get_available_providers() and returns a dict with device_type, ort_providers list, torch_device string, and insightface_ctx int.

Priority: CUDAExecutionProvider -> ROCMExecutionProvider -> OpenVINOExecutionProvider -> DmlExecutionProvider -> torch.xpu -> CPUExecutionProvider.

DEVICE env var overrides auto-detection ('cuda', 'rocm', 'openvino', 'cpu').

edge/src/detection/yolo_detector.py

YOLOv11 person detector + ByteTrack

CHANGE

device parameter now defaults to 'auto' -- reads torch_device from gpu.detect()

Uses ultralytics YOLO.track() which runs detection AND ByteTrack in one call. persist=True maintains track IDs across frames. PERSON_CLASS=0 filters to humans only.

edge/src/recognition/arcface.py

InsightFace ArcFace R100 face embedder

CHANGE

Now reads ort_providers and insightface_ctx from gpu.detect() instead of hardcoded ctx_id=0

detect_and_embed() returns list of (bbox, normed_embedding_512d, det_score). InsightFace's buffalo_l model includes SCRFD face detector + ArcFace embedder in one package.

edge/src/recognition/anti_spoof.py

ONNX liveness / anti-spoof model

BUG FIX

Hardcoded ['CUDAExecutionProvider', 'CPUExecutionProvider'] replaced with gpu.detect()['ort_providers']

Loads an ONNX model from model_path. check() returns (is_live: bool, score: float). If no model file found, returns 1.0 (pass-through). _infer() resizes face to 80x80, runs ONNX session, returns P(live) from softmax output.

edge/src/recognition/faiss_search.py

FAISS nearest-neighbour face search

build() creates an IndexFlatIP (inner product = cosine similarity on L2-normalised vectors). search() returns (employee_id, similarity_score). Returns (None, 0) if index is empty or score < threshold.

edge/src/pipeline/frame_processor.py

Main per-frame AI pipeline

BUG FIX

CRITICAL BUG FIX: duplicate 'snapshot_url' key in spoof_attempt event dict (second value silently overwrote first)

BUG FIX

snap variable was referenced before assignment in spoof branch -- now assigned before event dict

`_INFERENCE_EXECUTOR` is a module-level `ThreadPoolExecutor(max_workers=1)`. This serialises all ONNX/FAISS calls across all cameras since the models are not thread-safe. `process()` offloads to the executor via `run_in_executor()` so the asyncio event loop (and MJPEG server) stays responsive.

Per-track cooldown (`_cooldown` dict) prevents the same track from being processed multiple times. Per-track labels (`_track_labels`) make bounding box annotations persist between detections.

`edge/src/pipeline/event_publisher.py`

Posts events to backend with `EDGE_TOKEN` auth

SECURITY Added `EDGE_TOKEN` header: reads `os.getenv('EDGE_TOKEN')` and sends as `Authorization: Bearer`

`publish()` first posts to Redis `pub/sub` (`recognition:{tenant_id}`), then HTTP POSTs to backend `/attendance/event` with the `EDGE_TOKEN` header. Redis publish is fire-and-forget; HTTP POST logs errors but does not block the pipeline.

`edge/src/main.py`

Edge node entrypoint

NEW Calls `log_gpu_info()` at startup to log detected GPU type and ORT providers

CHANGE Passes `device='auto'` to all model constructors (was `device=os.getenv('DEVICE', 'cuda')`)

Loads config -> authenticates with backend -> loads cameras -> loads embeddings -> builds FAISS -> starts MJPEG server -> starts `FrameProcessors` per camera -> runs heartbeat + embedding-watch + camera-watch loops.

5. Dockerfiles

backend/Dockerfile

Production backend -- pre-bakes InsightFace buffalo_l at build time

Runs 'python -c from insightface...' during docker build to download the 500 MB model pack into the image layer. First build is slow; subsequent builds use the cached layer.

backend/Dockerfile.dev

Development backend -- skips model pre-download

NEW New file -- faster dev build (models download on first enrollment request)

edge/Dockerfile.edge.nvidia

NVIDIA CUDA edge (renamed from Dockerfile.edge)

Base: nvcr.io/nvidia/pytorch. Installs onnxruntime-gpu. Pre-downloads InsightFace and YOLO11s. Force-reinstalls numpy<2.0 to fix ABI conflict with NVIDIA's bundled OpenCV.

edge/Dockerfile.edge.amd

AMD ROCm edge -- NEW file

NEW New file

Base: rocm/pytorch:rocm6.2.4. Removes onnxruntime-gpu, installs onnxruntime-rocm from AMD's extra-index. Pre-downloads models using ROCMExecutionProvider.

edge/Dockerfile.edge.intel

Intel OpenVINO edge -- NEW file

NEW New file

Base: ubuntu:22.04 with python3.11. Installs openvino and onnxruntime-openvino. Models pre-downloaded with OpenVINOExecutionProvider.

edge/Dockerfile.edge.cpu

CPU-only edge (existing, unchanged)

Base: python:3.11-slim. Swaps onnxruntime-gpu for onnxruntime via sed on requirements.txt. Suitable for any machine.

6. Infrastructure Files

infra/nginx/nginx.conf

Nginx reverse proxy configuration

BUG FIX

CRITICAL FIX: Added resolver 127.0.0.11 (Docker DNS) + set \$upstream variables. Previously nginx crashed on startup if

All upstream hostnames are now in set \$var variables, resolved at request time by Docker's DNS server (127.0.0.11). /stream/ returns 502 gracefully when edge_node is offline.

docker-compose.dev.yml

Development compose -- NEW file

NEW

New file -- no NVIDIA GPU requirement, fast backend Dockerfile, nginx on port 80, edge_node optional via --profile edge

docker-compose.prod.yml

Production compose

CHANGE

Backend command overridden to --workers 2 for production throughput

CHANGE

NEXT_PUBLIC_* passed as build args to bake server URL into frontend

CHANGE

Edge node uses CPU Dockerfile by default; GPU via overlay files

docker-compose.prod.gpu.yml

NVIDIA GPU overlay

Overrides edge_node to use Dockerfile.edge.nvidia, DEVICE=cuda, and adds nvidia deploy.resources.reservations.devices.

docker-compose.prod.amd.yml

AMD ROCm overlay -- NEW file

NEW

New file

Mounts /dev/kfd and /dev/dri, adds user to render+video groups. Uses Dockerfile.edge.amd with DEVICE=rocm.

docker-compose.prod.intel.yml

Intel GPU overlay -- NEW file

NEW

New file

Mounts /dev/dri, adds user to render+video groups. Uses Dockerfile.edge.intel with DEVICE=openvino.

deploy.sh

Linux/Mac production one-click deploy

NEW

Full GPU detection: NVIDIA (nvidia-smi + nvidia-container-toolkit check), AMD (/dev/kfd + rocm-smi), Intel (lspci + /dev/dri)

NEW

Secret auto-generation: openssl rand-hex for keys, python3 base64.urlsafe_b64encode for Fernet key

NEW

--update flag preserves .env but updates server URL

NEW

--gpu flag forces specific GPU type

deploy.bat

Windows Server production one-click deploy

NEW

PowerShell-based secret generation using RandomNumberGenerator

NEW

GPU detection via WMI Win32_VideoController for AMD/Intel; nvidia-smi for NVIDIA

7. Security Architecture

Complete threat model

Threat	Mitigation
Forged JWT token	SECRET_KEY=random 64-char hex; startup refuses default in prod
Brute-force login	30 req/min/IP rate limit on /auth/login
Fake check-in events	EDGE_TOKEN Bearer auth on /attendance/event
API enumeration	Swagger disabled in production
Privilege escalation	JWT role claim validated against DB-backed role on every request
Cross-tenant access	Every query filters by tenant_id from JWT payload
Spoof attack (photo)	ONNX liveness model; score < 0.80 blocks recognition
Face data exposure	Embeddings encrypted at rest with Fernet key
Password brute-force	bcrypt with 12 rounds; rate limiting on login endpoint
Config data leak	GET /alerts/config requires admin role
Audit bypass	All user actions logged with IP + user agent to audit_logs table
Token replay	JWT expires in ACCESS_TOKEN_EXPIRE_HOURS (default 8 h)

RBAC enforcement points

Two independent enforcement points:

- Backend: every endpoint has a Depends(role_required('hr')) or similar -- 403 Forbidden if insufficient role
- Frontend: useRole() hook + can() function hide buttons and pages after hydration -- defence in depth, not a substitute for backend checks

8. Database Schema

All tables have a UUID primary key generated by the uuid-osp extension. pgvector extension installed via init.sql.

Table	Key columns
tenants	id, name, slug, plan, max_employees, max_cameras, is_active, settings(JSON)
branches	id, tenant_id, name, code, timezone, geo_lat, geo_lng, is_active
shifts	id, tenant_id, branch_id, name, start_time, end_time, grace_in_min, early_out_min, work_days(INT[])
employee_shifts	employee_id, shift_id, effective_from, effective_to (composite PK)
employees	id, tenant_id, branch_id, full_name, employee_code, email, phone, department, is_enrolled, is_blacklisted,
face_embeddings	id, employee_id, tenant_id, embedding(VECTOR(512)), quality_score, photo_angle
cameras	id, tenant_id, name, rtsp_url, direction, camera_role, camera_zone, is_restricted, fps_target, status
attendance_logs	id, tenant_id, employee_id, shift_id, attendance_date, check_in_at, check_out_at, status, is_late, late_by,
recognition_events	id, tenant_id, camera_id, employee_id, confidence, is_live, spoof_score, raw_event(JSON)
alerts	id, tenant_id, alert_type, severity, employee_id, camera_id, is_acknowledged, acknowledged_by
alert_configs	id, tenant_id, confidence_threshold, liveness_threshold, cooldown_minutes, loitering_threshold_min, admin
users	id, tenant_id, email, hashed_password, full_name, role, is_active, last_login_at
audit_logs	id, tenant_id, user_id, action, resource_type, resource_id, old_values(JSON), new_values(JSON), ip_addr
unknown_detections	id, tenant_id, camera_id, track_id, snapshot_url, confidence_best, detection_date

9. GPU Support Reference

Detection order in gpu.py

```

DEVICE env var override (cuda | rocm | openvino | cpu)
|
onnxruntime.get_available_providers()
  CUDAXecutionProvider      -> NVIDIA
  ROCMEExecutionProvider     -> AMD (Linux ROCm)
  OpenVINOExecutionProvider  -> Intel Arc / Iris / UHD
  DmlExecutionProvider       -> DirectML (Windows, any DirectX 12 GPU)
|
torch.xpu.is_available()    -> Intel IPEX XPU
|
CPUExecutionProvider       -> CPU fallback

```

Runtime graceful degradation

If a provider initialises but fails on first `run()` call, ONNX Runtime automatically falls through to the next provider in the list. For example, if `CUDAXecutionProvider` is listed but the GPU runs out of memory, the next inference call uses `CPUExecutionProvider`. No crash, no manual intervention needed.

Host requirements per GPU type

GPU	Host requirement	Docker mount
NVIDIA	nvidia-driver + nvidia-container-toolkit	--gpus all
AMD	ROCm drivers (Linux only)	/dev/kfd + /dev/dri
Intel	i915 or xe kernel module (standard)	/dev/dri
CPU	None	None

10. Extension Guide

Adding a new API endpoint

- 1. Add route function to appropriate backend/app/api/v1/*.py
- 2. Add role check: Depends(role_required('hr'))
- 3. Add Pydantic schema to backend/app/schemas/ if needed
- 4. Register new router file in main.py if creating a new file
- 5. Add TypeScript API method to frontend/src/lib/api.ts
- 6. Use from page component with useRole() for conditional UI

Adding a new dashboard page

- 1. Create frontend/src/app/dashboard/<name>/page.tsx
- 2. Import DashboardShell and useRole()
- 3. Add nav entry to NAV array in DashboardShell.tsx
- 4. If admin-only: add to the conditional Admin link block (not NAV array)

Adding a new alert type

- 1. Add to SEVERITY dict in alert_service.py
- 2. Call alerts.fire(tenant_id, 'new_type', message) from attendance_service.py
- 3. Add to TYPE_LABEL in alerts/page.tsx and security-alerts/page.tsx
- 4. Add to available actions list in rbac.py for audit filtering

Supporting a new GPU

- 1. Create edge/Dockerfile.edge.<name> with GPU-specific base image
- 2. Add detection logic in edge/src/utlis/gpu.py detect() function
- 3. Create docker-compose.prod.<name>.yml with required device mounts
- 4. Add GPU detection to deploy.sh and set GPU_OVERRIDE variable
- 5. Add make deploy-<name> target to Makefile